On the criteria to be used in decomposing systems into modules

SEPTEMBER 5, 2016JULY 31, 2017
On the criteria to be used in decomposing systems into modules (http://repository.cmu.edu/cgi/viewcontent.cgi?article=2979&context=compsci) *David L Parnas, 1971*

Welcome back to a new term of The Morning Paper! I thought I'd kick things off by revisiting a few of my favourite papers from when I very first started this exercise just over two years ago. At that time I wasn't posting blog summaries of the papers, so it's nice to go back and fill in that gap (blog posts started in October of 2014). Plus, revisiting some of the classics once every couple of years seems like a good idea – changing external circumstances can make them feel fresh again every time you read them.

Today's choice is a true classic, "On the criteria to be used in decomposing systems into modules." If we give that title a slight twist – "On the criteria to be used in decomposing systems into *services*" – it's easy to see how this 45-year old paper can speak to contemporary issues. And from the very first sentence of the abstract you'll find some shared goals with modern development: "This paper discusses modularization as a mechanism for improving the flexibility and comprehensibility of a system while allowing the shortening of its development time." Flexibility (we tend to call it agility) and faster development times remain top of mind today. Comprehensibility less so, but perhaps we should be paying more attention there? Say you're all bought into cloud native microservices based architectures, is splitting your system up into multiple independent services going to help you achieve your goals? Parnas brings a keen insight:

> The effectiveness of a "modularization" is dependent upon the criteria used in dividing the system into modules.

A timely reminder that it's not simply about having lots of small modules, a large part of the success or otherwise of your system depends on *how you choose to divide the system into modules in the first place*. When Parnas talks about a "module" in the paper, his definition is a *work assignment* unit, rather than a subprogram unit.

Parnas sets out three expected benefits of modular programming. We can look at those through the lens of microservices too:

1. Development time should be shortened because separate groups can work on each module (microservice) with little need for communication.
2. Product flexibility should be improved – it was hoped that it would be possible to make quite drastic changes or improvements in one module (microservice) without changing others.
3. Comprehensibility – it was hoped that the system could be studied a module (microservice) a[illegible]ne with the result that the whole system could be better designed because it was better understood.

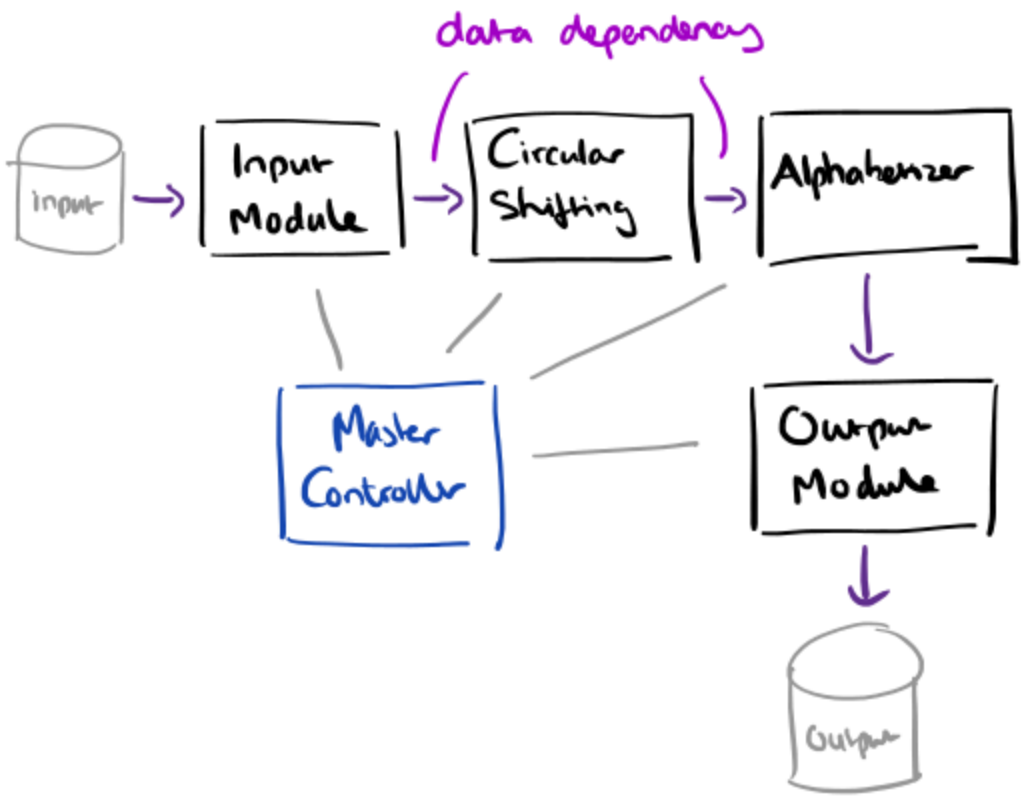
Different ways of dividing the system into modules bring with them different communication and coordination requirements between the individuals (or teams) working on those modules, and help to realise the benefits outlined above to greater or lesser extents.

The famous example at the heart of the paper is the development of a system to generate a 'KWIC' index given an ordered set of lines as input. Any line can be "circularly shifted" by repeatedly removing the first word and adding it to the end of the line. The system outputs a listing of all circular shifts of all lines in alphabetical order. Parnas acknowledges this is somewhat of a toy example, but "we shall go through the exercise of treating this problem as if it were a large project."

Two decompositions are examined. In the first decomposition, each major step or task in the processing workflow is made into its own independent module (service). This leads to five modules:

1. An input module which reads data lines from the input medium
2. A circular shifter
3. An 'alphabetizer' (sorter)
4. An output module which creates a nicely formatted output
5. A master control module which sequences the other four.

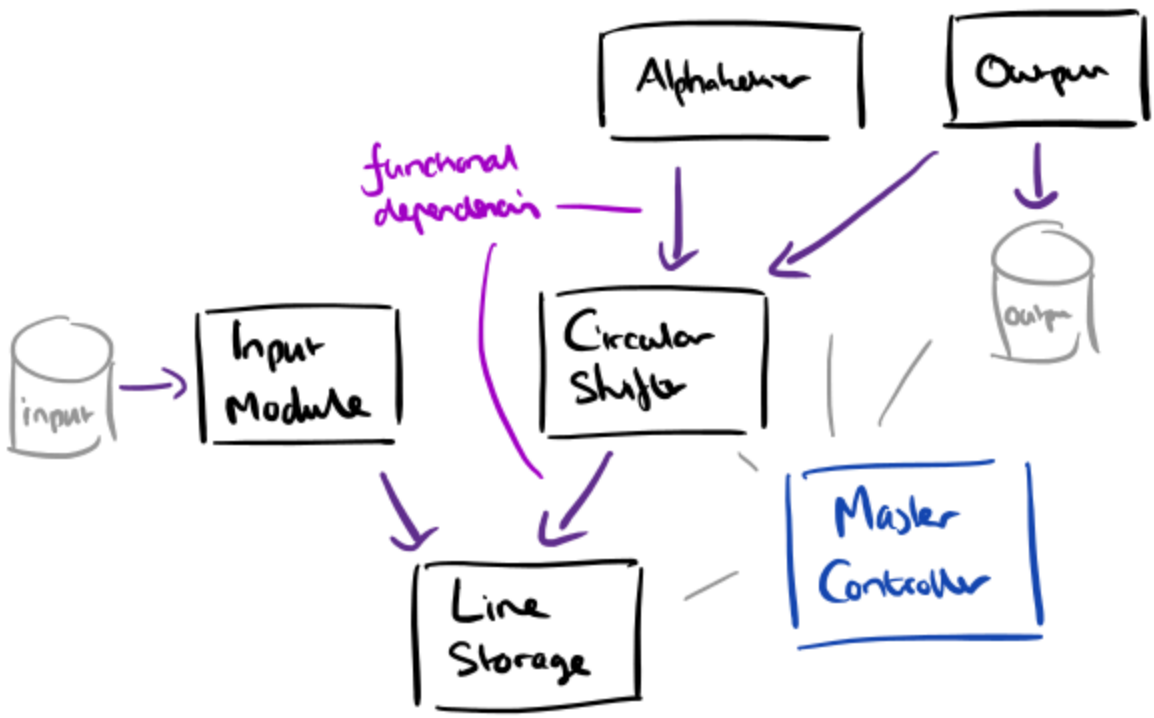
Modules 1 through 4 are operating on shared data structures in memory.



> This is a modularization in the sense meant by all proponents of modular programming. The system is divided into a number of relatively independent modules with well defined interfaces; each one is small enough and simple enough to be thoroughly understood and well programmed. Experiments on a small scale indicate that this is approximately the decomposition which would be proposed by most programmers for the task specified.

The second decomposition looks similar on the surface:

1. A line storage module with routines for operating on lines
2. An input module as before, but which calls the line storage module to have lines stored internally
3. A circular shifter, which shifts lines using the line storage module
4. An alphabetizer
5. An output module which builds upon the circular shifter functions
6. A master control module.



This decomposition however, was created on the basis of *information hiding*.

> There are a number of design decisions which are questionable and likely to change under many circumstances… It is by looking at changes such as these that we can see the differences between the two modularizations.

In the first decomposition, many changes (for example, the decision to have all lines stored in memory) require changes in every module, but with the second decomposition many more potential changes are confined to a single module.

Furthermore, in the first decomposition the interfaces between modules are fairly complex formats and represent design decisions that cannot be taken lightly. "The development of those formats will be a major part of the module development and that part must be a joint effort among the several development groups." In the second decomposition the interfaces are simpler and more abstract leading to faster independent development of modules.

Regarding comprehensibility, in the first decomposition the system can only really be understood as a whole, "it is my (Parnas's) subjective judgement that this is not true in the second modularization."

> Every module in the second decomposition is characterized by its knowledge of a design decision which it hides from all others. Its interface or definition was chosen to reveal as little as possible about its inner workings.

There's a potential drawback to the second decomposition though, which is even more important when packaging modules as independent services:

> If we are not careful, the second decomposition will prove to be much less efficient. If each of the "functions" is actually implemented as a procedure with an elaborate calling sequence there will be a great deal of such calling due to the repeated switching between modules. The first decomposition will not suffer from this problem because there is a relatively infrequent transfer of control between the modules.

To avoid this overhead Parnas recommends a tool that enables programs to be written as if the functions were subroutines, but *assembled* via whatever mechanism is appropriate. This is more challenging in a microservices world!

In conclusion, while much attention is given to the need to divide a system into modules (microservices), much less attention has been given to the *criteria* by which we decide on module boundaries. As Parnas shows us, it might be a good idea to think about those criteria in your next project, as they have a strong influence on development time, system agility, and comprehensibility.

> We have tried to demonstrate by these examples **that it is almost always incorrect to begin the decomposition of a system into modules on the basis of a flowchart**. We propose instead that one begins with a list of difficult design decisions or design decisions which are likely to change. Each module is then designed to hide such a decision from the others. Since, in most cases, design decisions transcend time of execution, modules will not correspond to steps in the processing…

SOFTWARE ENGINEERING

23 thoughts on "On the criteria to be used in decomposing systems into modules"

1. **Jedidja**
   SAYS:
   SEPTEMBER 5, 2016 AT 10:29 AM
   Very interesting stuff; seems to be a pre-cursor to Juval Lowy's thoughts on decomposition based on volatility rather than functionality. He readily admits that this thinking has been around for 40+ years, yet very few of us know about it or practice it. Thanks for sharing this paper!

2. **Alexandre Fruchaud (@AlexThep)**
   SAYS:
   SEPTEMBER 5, 2016 AT 11:02 AM
   Great article ! Love it. Really useful way to think about decomposition.

3. **vyodaiken**
   SAYS:

SEPTEMBER 6, 2016 AT 8:07 PM
Click to access win_pu.pdf

4. **lliamander**
SAYS:
SEPTEMBER 6, 2016 AT 11:09 PM
Excellent article. Here are a couple thoughts that occurred to me during reading:

– Of the benefits that Parnas hoped to achieve through modularization, it seems that the latter two are prerequisites for the first. That is, in order to be able to delegate module development to different teams, each team must be able to work on their own module without interference from other module teams (the second benefit) and it must be possible for senior developers to conceptualize the whole system in order to coordinate the work of the module teams (the third benefit).

– Considering the example of changing the Line Storage module, you can make the modules even further decoupled. Simply have some sort of Line Storage interface/protocol that can be implemented by multiple modules, allowing the Master Controller to select the appropriate implementation at runtime (i.e. Dependency Inversion + Dependency Injection).

– In general, I think that mastering the complexity of software systems as whole systems (i.e. realizing the third benefit) involves a careful study of the protocols/interfaces through which modules communicate and leveraging the algebraic composition of modules [0].

– You correctly identify this as the money quote: "…it is almost always incorrect to begin the decomposition of a system into modules on the basis of a flowchart. We propose instead that one begins with a list of difficult design decisions or design decisions which are likely to change." I think this juxtaposes nicely with the idea that Agile development is intended to reduce the cost of making decisions later in the development process.

[0] https://awelonblue.wordpress.com/2015/12/09/out-of-the-tarpit/

5. **Zteve**
SAYS:
SEPTEMBER 7, 2016 AT 11:57 AM
This is another timeless classic; a good choice to revisit occasionally. The actual paper details some of the thinking about hiding a design decision and is worth a read.

Notice DESIGN —up front— ah, bliss.

6. Pingback: Give me four links to teach the children and the seed I have sown will never be uprooted. - Vladimir Lenin - Magnus Udbjørg
7. Pingback: Worth Reading: On decomposing systems - 'net work
8. **David Weiss**
SAYS:
SEPTEMBER 21, 2016 AT 2:32 PM
The follow-up paper, "The Modular Structure of Complex Systems," Parnas, Clements, Weiss, is also well-worth reading. The explanation of information hiding and modularity is more detailed. The example shows the application of the ideas to a real avionics system.

9. **David Lorge Parnas**
SAYS:

SEPTEMBER 28, 2016 AT 6:28 AM
If you like those papers, you might also like:

228. Parnas, D.L., "The Secret History of Information Hiding", Software Pioneers: Contributions to Software Engineering, Manfred Broy and Ernst Denert (Eds.), Springer Verlag, Berlin – Heidelberg, 2002, 399-409, ISBN 3-540-43081-4.

10. **parnas2014**
SAYS:
SEPTEMBER 28, 2016 AT 6:29 AM
If you like those papers, you might also like:

228. Parnas, D.L., "The Secret History of Information Hiding", Software Pioneers: Contributions to Software Engineering, Manfred Broy and Ernst Denert (Eds.), Springer Verlag, Berlin – Heidelberg, 2002, 399-409, ISBN 3-540-43081-4.

11. Pingback: Program development by stepwise refinement | the morning paper
12. Pingback: Information distribution aspects of design methodology | the morning paper
13. Pingback: Why Networking is Critical to Serverless | Atomic Energy
14. Pingback: A design methodology for reliable software systems | the morning paper
15. Pingback: Designing software for ease of extension and contraction | the morning paper
16. Pingback: So that was 2016 | the morning paper
17. Pingback: Professional Development 9/04/2017 – 9/10/2017 – The Software Mentor
18. Pingback: The paradigms of programming | the morning paper
19. Pingback: Reading List March 2018 | Jega's Blog
20. Pingback: Programming paradigms for dummies: what every programmer should know | the morning paper
21. Pingback: Microservices masterclass – Nikhil Kuriakose
22. Pingback: How do committees invent? – the morning paper
23. Pingback: On the importance of design in software - Paul Stephen Borile

Comments are closed.

BLOG AT WORDPRESS.COM.